Affordable Mistakes: Severity-Aware Multiparty Session Types for Agents that Choose Wrongly*

Anonymous author
Anonymous affiliation

Anonymous author
Anonymous affiliation

Abstract

Multiparty session types guarantee that well-typed participants interact correctly. A participant that was instructed rather than compiled — a language-model agent, say — is not such a participant: at a branch point where it should select A it may select B. Forbidding this is impossible, and would be useless if it were possible: an agent that may never err may never act. We change what the type system is for. It does not prevent wrong choices; it prevents *catastrophic* ones, and reports the rest as risk. The fault model is the participant's *own selection at a protocol branch*, defined through the branch guards that established design-by-contract session types already carry; the consequence of a wrong selection is classified by *outcome* against a STRIPS-style world — goal still reachable, goal lost but nothing harmed, or hazard reachable. *Failure is not disaster*, and a discipline that conflates them is unusable. Cascading is captured by a misselection budget k, giving a purely possibilistic notion of k-misselection tolerance with no probabilities anywhere. A single syntax-directed rule, T-Choice-Safe, is *sound and complete* for k-bounded hazard-freedom, and — the property that makes it a type system rather than a model checker in disguise — sequential composition is typed *modularly* against an interface. All results are mechanized in Coq, axiom-free, for the finite fragment. A worked instance is 0-tolerant but not 1-tolerant and its repair is verified. One design expectation was refuted by the mechanization: tolerance is *anti*-monotone in the capability context, a formal argument for least privilege.

2012 ACM Subject Classification Theory of computation → Type structures; Theory of computation → Process calculi; Software and its engineering → Software verification

Keywords and phrases Session types, Multiparty, Behavioural types, Safety, Irreversibility, Fault tolerance, Mechanized metatheory

Digital Object Identifier 10.4230/LIPIcs.ECOOP..0

Supplementary Material Anonymous supplementary material

Funding Anonymous funding

Acknowledgements Anonymous acknowledgements

WIP: Status. Sections 5–8 are backed by `proof/Severity.v` (Coq 8.18, stdlib only, axiom-free; 18 theorems), for the *finite* fragment — the scope of the existing base mechanization. Recursion is future work. No experimental numbers are claimed. The predecessor draft (a mode/taint/grade "deviation layer") was withdrawn after its own mechanized audit found it vacuous; that audit is retained in `proof/AUDIT.md` and summarized in Section 11.

1 Introduction

Multiparty session types (MPST) guarantee communication safety, session fidelity and deadlock-freedom for systems whose participants *are* their local types [1]: the code was

* WORKING DRAFT — not for submission. Base rules follow `paper/tas.tex`, which remains authoritative for Section 3.

Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

checked, so every run is a compliant run. A participant that was instructed rather than compiled breaks that hypothesis in a specific way. At a choice point it may take branch B where the situation demanded branch A — not by violating the protocol (both branches are in the type) but by judging the situation wrongly. This is the ordinary condition of a language-model agent occupying a role, and it is the reason such agents are employed: their latitude.

The tempting response is to make the type system prevent it. That is doubly wrong. It is impossible, because the judgement in question is the participant's, not the protocol's; and it is undesirable, because a participant that may never choose wrongly is one that may never choose. What is needed is not prevention but *triage*.

The reframing. Do not ask whether the participant complied. Ask *what a wrong choice costs*. Three answers matter and the middle one is what existing disciplines cannot express (Figure 1):

- Benign — a detour; the goal is still reachable.
- Futile — the goal is lost, but nothing is harmed. *Failure is not disaster.*
- Catastrophic — a hazard becomes reachable.

A system that blocks Futile is useless; one that permits Catastrophic is dangerous. Separating them is the whole job. Our thesis line: *session types say what may happen; this says what you can survive.*

What is and is not new here. We are explicit, because the ingredients are old and the combination is not. Branch guards on global types are design-by-contract MPST [2, 4, 5], and violation of a guard on selection is already the alarm condition of the monitoring line [3]; we do not claim them. Deciding whether a goal is still reachable is dead-end detection in classical planning [28, 29], and deciding whether an effect can be undone is undoability checking [30, 31]; we do not claim those either. Bounded fault tolerance is classical, in planning [26, 27] and in MPST with crash-stop failures [8, 9]. A syntax-directed type system that is *equivalent* to a model checker is a known and valuable template [21, 22]. What we claim is one thing:

> *The first behavioural type theory whose fault model is the participant's own selection at a protocol branch point — misselection graded by an explicit budget, with typing a syntax-directed, mechanized, exact characterization of hazard-freedom under at most k misselections, composed modularly — and whose analytic payload is that pairing the protocol with a world state classifies each deviation by outcome rather than merely detecting it.*

Why this needs a world. Severity is a function of the *node and the world state*, not of the label. "Skip validation" is benign before a payment is authorized and catastrophic after. Plain MPST cannot state the difference because it has no world. The capability-guarded, goal-marked discipline we build on does, and that is what earns it the right to answer the question. Because effects are STRIPS-style, catastrophe has a computable signature: an atom deleted by a returns only if some available capability adds it, so catastrophe is *irreversibility conjoined with unrecoverability* — the crossing of a *point of no return* — decided by the delete-list reachability search the underlying checker already runs.

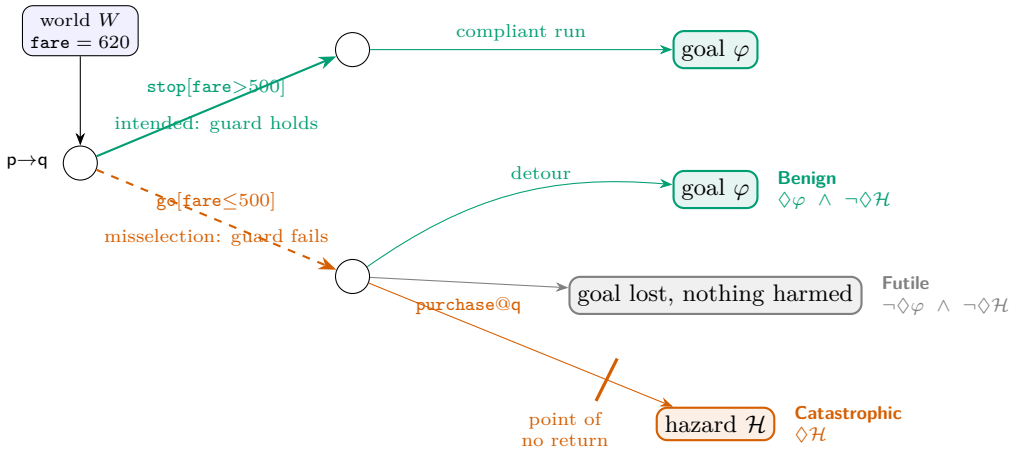

Figure 1 The idea. At a guarded choice in world W, the branch whose guard holds is *intended*; taking the other is a *misselection*. Its severity is a property of the residual: Benign if the goal is still reachable, Futile if the goal is lost but no hazard is reachable, Catastrophic if a hazard is reachable — typically by crossing a *point of no return*, an irreversible capability after which the goal cannot be recovered.

Contributions.

C1 *Misselection and its severity* (§4, §5): guard failure at a branch as the fault model; a budgeted semantics in which compliant progress is free and a wrong branch costs one unit; the Benign/Futile/Catastrophic partition (✓ `severity_disjoint`, ✓ `severity_exhaustive`).

C2 *T-Choice-Safe, sound and complete* (§6–7): an exact syntax-directed characterization of k-bounded hazard-freedom (✓ `TC_sound`, ✓ `TC_complete`, ✓ `TC_exact`). A typing failure is therefore a genuine risk, never an artifact of conservative rules.

C3 *Modularity* (§8): sequential composition is typed against an interface, not a product state space (✓ `TC_seq`, ✓ `TC_seq_interface`). This is our answer to "why a type system".

C4 *Mechanization and a refuted expectation*: an axiom-free Coq development, and the discovery that tolerance is *anti*-monotone in the capability context (✓ `tolerance_antitone_in_ctx`): more tools can only make a participant less tolerant of its own mistakes.

Organization. §2 walks the running example and the architecture. §3 fixes notation. §4–§7 give the fault model, severity, the rule, and the metatheory. §8 argues why this is a type system. §9 treats repairs briefly. §10 positions the work against runtime compliance monitoring. §11 reports the refuted predecessor. §12, §13 and §14 close.

2 Overview

Running example. Two roles: a planner `p` and a worker `q` holding a purchase tool. Atoms `verified` and `booked`; `verify` sets the first, `purchase` sets the second and is externally visible. The hazard is booking without verification, $\mathcal{H} \equiv \texttt{booked} \wedge \neg\texttt{verified}$. The protocol offers two branches, each carrying the guard that makes it the intended one:

$$G_{\mathsf{bad}} \;=\; \mathtt{p}\to\mathtt{q} : \{\; \mathtt{safe}[\top].\ \mathtt{verify@q}.\ \mathtt{purchase@q}.\ \mathsf{End}\ ,\quad \mathtt{fast}[\bot].\ \mathtt{purchase@q}.\ \mathsf{End}\;\}$$

The fast path's guard never holds, so taking it is precisely a misselection. What the analysis reports, each part machine-checked:

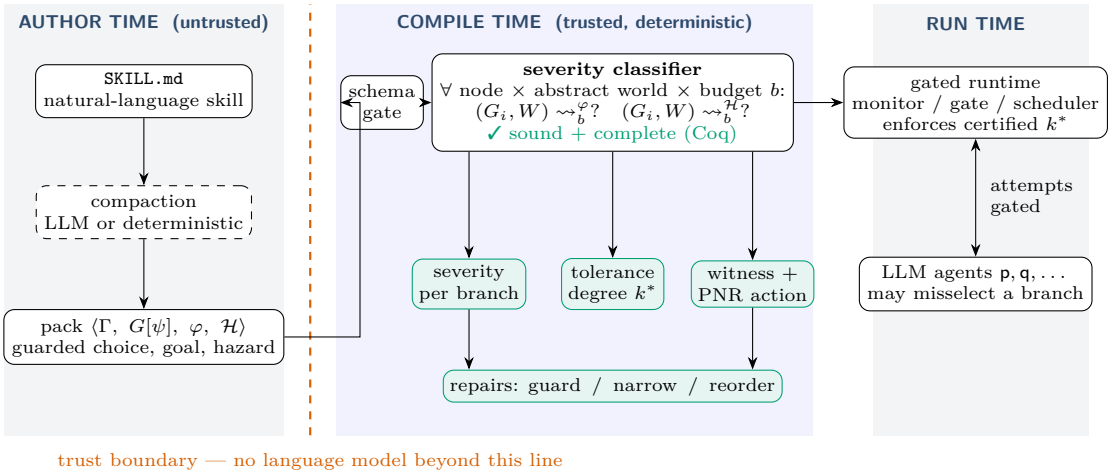

Figure 2 Architecture. Compaction from prose to a pack is untrusted and may use a language model; nothing beyond the trust boundary does. The severity classifier runs the discipline's existing reachability search pointwise at every branch of every choice, at every abstract world and budget, and emits a per-branch severity, the tolerance degree k^*, a witness naming the point-of-no-return action, and repair suggestions. The certified pack drives the gated runtime that refuses out-of-protocol attempts.

- G_{bad} *is* 0-tolerant (✓ `Gbad_is_0_tolerant`): if the participant never misselects, no hazard arises. Classical MPST stops here and calls the protocol fine.
- G_{bad} is *not* 1-tolerant (✓ `Gbad_not_1_tolerant`): one wrong choice books without verifying. No discipline without a world and a misselection notion can produce this verdict.
- Narrowing the offer yields G_{good}, k-tolerant for every k (✓ `Ggood_is_k_tolerant`), as an instance of the general narrowing theorem (✓ `Ggood_by_narrowing`).

The headline number for a protocol is its *tolerance degree* k^*: the largest k it survives. "This skill is 0-tolerant: one wrong branch at step 3 fires an irreversible purchase" is a finding an engineer can act on. "Impossible" is not.

Architecture. Figure 2 places the analysis. It is a compile-time check on the pack, before any participant runs, and its trusted core contains no language model — the same trust boundary as the base discipline. The runtime gate is inherited unchanged: it refuses *out-of-set* attempts (labels the protocol does not offer) before delivery, so those change no state and are safe by construction. The interesting deviation, and the only one this paper analyses, is *in-set*: a branch the type permits and the situation forbids.

3 The Base Discipline

We build on a goal-marked, capability-guarded synchronous multiparty discipline; `tas.tex` is authoritative and this section only fixes notation. Predicates $p \in \mathsf{Pred}$, numeric variables $x \in \mathsf{Var}$, roles p, q, capabilities $a \in \mathsf{Cap}$, labels $\ell \in \mathsf{Lab}$. The state logic is quantifier-free linear integer arithmetic with uninterpreted booleans; a world $W = \langle B, N \rangle$ valuates it. A capability context maps each possessed tool to a guarded STRIPS effect $\Gamma(a) = \langle pre_a, \mathit{eff}_a \rangle$, $\mathit{eff}_a = \langle Add_a, Del_a, Asg_a, Nd_a \rangle$; $a \notin \mathrm{dom}(\Gamma)$ means the tool does not exist. Firing is WORLD-ACT: $\langle W \rangle\, a\, \langle W' \rangle$ iff $W \vDash pre_a$ and $W' \in \mathrm{apply}(\mathit{eff}_a, W)$. Global types are coinductive

regular trees

$$G \;::=^{coind}\; \mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I} \;\mid\; a@\mathsf{p}.\, G \;\mid\; \checkmark\varphi.\, G \;\mid\; \mathsf{End}$$

typed directly against a session with no local types, projection, or separate subtyping [16, 17]; achievability is existential may-reachability of a goal-satisfying configuration. Everything below adds one annotation to choice and one counter to the semantics.

4 Misselection and the Budget

▶ **Definition 1** (Guarded choice). *A choice node carries, per branch, the predicate that makes that branch the intended one:* $\mathsf{p}{\to}\mathsf{q} : \{\ell_i[\psi_i].\, G_i\}_{i\in I}$. *In world* W, *branch* i *is* intended *if* $W \vDash \psi_i$; *taking a branch with* $W \nvDash \psi_i$ *is a* misselection.

The syntax is that of asserted global types [2], whose well-formedness conditions exist precisely to guarantee that a compliant participant can always pick a satisfied branch. We keep the syntax and drop the guarantee: the guards are not a constraint on a well-typed implementation but the *definition of wrong*. They are already present, informally, in the natural-language skill ("if the fare is under $500, book it") and formally in the biconditional choice guards of protocol-compiled agent runtimes.

▶ **Definition 2** (Budgeted reachability). $(G, W) \rightsquigarrow_b^{\mathcal{H}}$ *holds when a hazard state is reachable from* (G, W) *using at most* b *misselections. The rules (Coq:* `reach_haz`*) are the head rules of the base semantics, plus*

$$\textit{RH-Comm-Ok}\ \frac{\ell_i[\psi_i].G_i \in \mathit{brs} \qquad W \vDash \psi_i \qquad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathsf{p}{\to}\mathsf{q} : \mathit{brs}, W) \rightsquigarrow_b^{\mathcal{H}}}$$

$$\textit{RH-Comm-Dev}\ \frac{\ell_i[\psi_i].G_i \in \mathit{brs} \qquad W \nvDash \psi_i \qquad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathsf{p}{\to}\mathsf{q} : \mathit{brs}, W) \rightsquigarrow_{b+1}^{\mathcal{H}}}$$

Compliant progress preserves the budget; a misselection consumes one unit.

▶ **Definition 3** (k-misselection tolerance). G *is* k-misselection-tolerant *for hazard* $\mathcal{H}$ *under* Γ *from* W_0 *if* $\neg\,(G, W_0) \rightsquigarrow_k^{\mathcal{H}}$.[1] 0*-tolerance is classical MPST safety;* 1*-tolerance says no single wrong choice is fatal.*

The budget is possibilistic. There are no probabilities in this paper: the budget is a design parameter of the deployment, not a measurement of the participant, and the theorems below hold for every participant that misselects at most k times, however it does so.

5 Severity

Let φ be the goal and $\mathcal{H}$ the hazard, and write $\rightsquigarrow_b^{\varphi}$ for budgeted goal reachability, defined identically.

[1] We avoid two established names. In MPST, *k-safety* is a buffer bound in k-multiparty compatibility [14]; in hyperproperties it is a property of k-tuples of traces [15]. *k-resilient* is used for plans surviving k action failures [26]; our fault is the participant's own selection and our target is a hazard rather than the goal.

▶ **Definition 4** (Severity). *For a residual (G, W) at budget b:*

$$\mathrm{sev}_b(G,W) = \begin{cases} \textit{Catastrophic} & \textit{if } (G,W) \rightsquigarrow_b^{\mathcal{H}} \\ \textit{Futile} & \textit{if } \neg(G,W) \rightsquigarrow_b^{\mathcal{H}} \textit{ and } \neg(G,W) \rightsquigarrow_b^{\varphi} \\ \textit{Benign} & \textit{if } \neg(G,W) \rightsquigarrow_b^{\mathcal{H}} \textit{ and } (G,W) \rightsquigarrow_b^{\varphi} \end{cases}$$

▶ **Theorem 5** (Severity is a partition). *The classes are pairwise disjoint, and exhaustive whenever the two reachability questions are decidable, which the QF-LIA fragment supplies.* ✓ `severity_disjoint`, ✓ `severity_exhaustive`

The content is the separation of Futile from Catastrophic: most wrong choices lose the goal and harm nothing, and a discipline that refuses them is not safe but merely unusable. In model-based safety assessment [34], severity is supplied by the analyst; here it is computed from goal reachability, which is the gap this section fills.

▶ **Definition 6** (Point of no return). *(G, W) is a* point of no return *for φ if $\neg(G,W) \rightsquigarrow_b^{\varphi}$. An action $a@\mathsf{p}$ is* goal-destroying *at (G, W) if the goal is reachable before it fires and at no successor.*

Because effects are STRIPS, an atom in Del_a is restored only by some b with that atom in Add_b; if Γ contains no such b the loss is permanent, and goal-destruction is decided by *establisher closure* — the protocol-independent refutation the reference checker already computes. This is dead-end detection [28, 29] and undoability checking [30] lifted to a protocol with roles and choice points; we note that unrestricted reversibility checking is harder than planning [31], and that our decidability rests on the widened finite-range fragment the checker already commits to, not on any new argument.

6 The Typing Rule

The judgment $\vdash_b G, W$ reads: *from (G, W), no hazard arises under any run with at most b misselections.*

$$\textsc{T-End}\ \frac{W \not\models \mathcal{H}}{\vdash_b \mathsf{End}, W} \qquad \textsc{T-Goal}\ \frac{W \not\models \mathcal{H} \qquad \vdash_b G, W}{\vdash_b \checkmark\varphi.G, W}$$

$$\textsc{T-Act}\ \frac{W \not\models \mathcal{H} \qquad \forall W'.\ \langle W\rangle\, a\, \langle W'\rangle \implies \vdash_b G, W'}{\vdash_b a@\mathsf{p}.G,\ W}$$

$$\textsc{T-Choice-Safe}\ \frac{\begin{array}{c} W \not\models \mathcal{H} \\ \forall i \in I.\ \ W \models \psi_i \implies \vdash_b G_i, W \\ \forall i \in I.\ \ W \not\models \psi_i \implies \forall c.\ b = c{+}1 \implies \vdash_c G_i, W \end{array}}{\vdash_b\ \mathsf{p}{\to}\mathsf{q} : \{\ell_i[\psi_i].G_i\}_{i\in I},\ W}$$

T-Choice-Safe is the formal content of "affordable mistake". An intended branch is checked at the same budget; a misselectable branch at one unit less, and not at all when the budget is exhausted. The rule does not forbid choosing B; it requires that choosing B, within the error budget the deployment tolerates, cannot reach a hazard.

Not resource-aware typing. The budget resembles the potential of resource-aware session types [23, 24] and the indices of graded modal types [25], and the resemblance is misleading.

Potential is consumed by the program's own execution and must be non-negative at the end; exhaustion is a type error. Our budget is consumed by an *adversarial* choice the program does not make, so the rule quantifies over deviations rather than amortising cost, and exhaustion is *unconstrained* — a branch beyond the budget is simply not the type system's business. That is a universal quantifier over an environment, not an accounting of a resource.

7 Metatheory

All results here and in §8 are mechanized in Coq 8.18, stdlib only, axiom-free (`proof/Severity.v`; `Print Assumptions` harness in `check_sev.v`), for the finite fragment. Recursion is future work.

▶ **Theorem 7** (Soundness). *If* $\vdash_k G, W$ *then no run from* (G, W) *with at most* k *misselections reaches a hazard.* ✓ `TC_sound`

▶ **Theorem 8** (Completeness). *If no run from* (G, W) *with at most* k *misselections reaches a hazard, then* $\vdash_k G, W$. ✓ `TC_complete`

▶ **Corollary 9** (Exactness). $\vdash_k G, W \iff \neg(G, W) \rightsquigarrow_k^{\mathcal{H}}$; *hence a branch is Catastrophic iff its residual is untypable.* ✓ `TC_exact`, ✓ `catastrophe_implies_untypable`, ✓ `untypable_implies_catastrophe`

▶ **Theorem 10** (Budget monotonicity). $\vdash_k G, W$ *and* $k' \le k$ *imply* $\vdash_{k'} G, W$. ✓ `tolerance_downward_closed`

▶ **Theorem 11** (Anti-monotonicity in the capability context). *Reachability is monotone in* Γ*; hence if* $\Gamma \subseteq \Gamma'$ *and* G *is* k*-misselection-tolerant under* Γ'*, it is* k*-misselection-tolerant under* Γ*, and not conversely.* ✓ `reach_monotone_in_ctx`, ✓ `tolerance_antitone_in_ctx`

Theorem 11 refuted a design expectation. We had predicted severity would be monotone-*decreasing* in Γ, reasoning that more tools give more ways to recover. That is false: the same tools open new paths to the hazard, and reachability is monotone in both directions at once. Granting a participant a capability can only lower its tolerance degree. This is the formal argument for least-privilege capability contexts, and a better result than the one we expected.

8 Why a Type System, and Not a Model Checker

Corollary 9 invites an objection: the judgment is definitionally bounded reachability, so the rules are a decision procedure in disguise. The objection is old — a type system provably equivalent to a model checker is a known template [21, 22], and Naik and Palsberg already gave the standard reply, that types explain why a program is *accepted* where a model checker explains why one is *rejected*. We do not rest on that reply. We rest on a theorem.

Write $G_1; G_2$ for sequential composition (substituting G_2 for End in G_1), and let $ends_b(G_1, W)$ be the set of (b', W') at which G_1 can terminate from budget b at W — its *interface*.

▶ **Theorem 12** (Modular composition). *If* $\vdash_k G_1, W$ *and* $\vdash_{b'} G_2, W'$ *for every* $(b', W') \in ends_k(G_1, W)$*, then* $\vdash_k G_1; G_2, W$. ✓ `TC_seq`. *In interface form: for any* I *with* $ends_k(G_1, W) \subseteq I$*, it suffices that* $\vdash_{b'} G_2, W'$ *for all* $(b', W') \in I$. ✓ `TC_seq_interface`

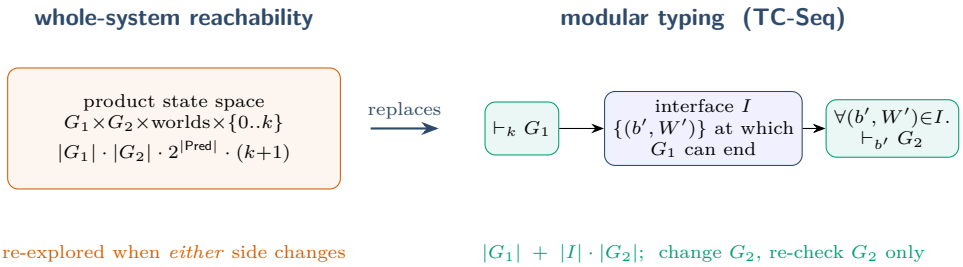

Figure 3 Modularity. A whole-system check of $G_1; G_2$ explores the product of both protocols with the world and the budget, and re-explores it when either side changes. TC-Seq types G_2 only at the *interface* G_1 exposes — the (remaining budget, world) pairs at which it can end — so changing G_2 re-checks G_2 alone.

This is the distinction that matters (Figure 3). Bounded reachability over $G_1; G_2$ is a whole-system exploration of $|G_1| \cdot |G_2| \cdot 2^{|\text{Pred}|} \cdot (k{+}1)$ states, re-run when either side changes. Theorem 12 checks G_2 once against a declared invariant I — a published contract on the boundary — and never looks inside G_1. That is precisely the axis on which MPST is currently being argued: the synthetic reconstruction of MPST [18] observes that projection-based techniques are compositional but limited, while the expressive recent techniques rely on whole-system model checking that scales poorly. Our answer to "why a type system" is that the budget becomes a contract on an interface, which a model-checking run cannot be. The community has already accepted this move: MPST typing parametric in a model-checked safety property [8] was extended at ECOOP with a Distinguished Paper [9].

WIP: Obligation. The rebuttal is rhetoric until demonstrated: the evaluation (§12) must show a composed protocol on which the product model checker times out and the modular derivation does not.

9 Repairs

When a branch is Catastrophic at the intended budget, four transformations make the mistake affordable: *guard* (insert a ✓φ or attestation before the irreversible branch), *compensate* (add a recovery path), *narrow* (remove the branch so the gate refuses it), and *reorder* (move the irreversible action after the validation, the one transformation that raises k). All have ancestors: amending asserted choreographies [35], interactional exceptions [36], supervisory control and shield synthesis [37, 38, 39], and choreography repair [40, 41]. We claim none of them as mechanisms; we record the one theorem we need.

▶ **Theorem 13** (Narrowing is sound). *Removing branches from a choice node preserves well-typedness at the same budget.* ✓ `repair_narrow_sound`

Narrowing is a supervisory-control disabling of a controllable event; its soundness is unsurprising and it is the repair the tool proposes first. *Reorder* is the interesting one, because it is the only repair that uses the world-state analysis rather than control-flow surgery: it is well-defined exactly when the point-of-no-return action commutes with the validation, and its soundness is the principal outstanding mechanization.

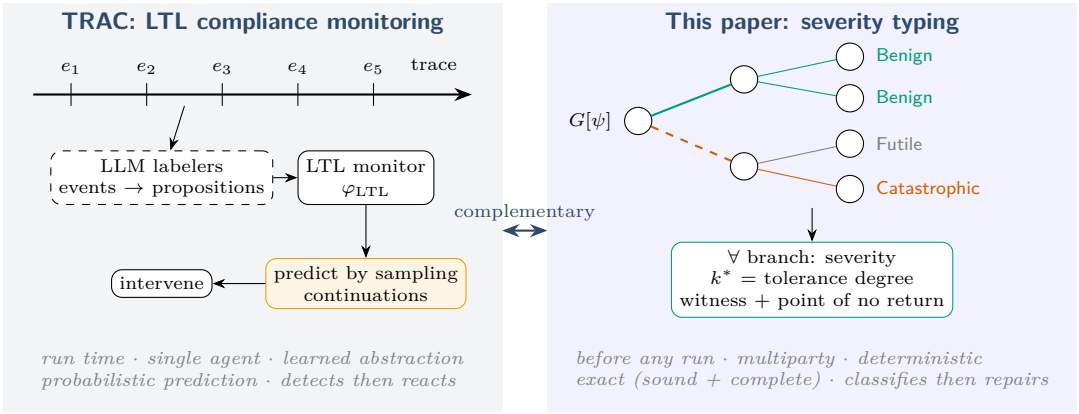

Figure 4 Two answers to "will the agent misbehave?". Left: TRAC observes a single agent's trace at run time, abstracts events to propositions with language-model labelers, checks an LTL constraint, predicts violations by sampling continuations, and intervenes. Right: this paper classifies every branch of a multiparty protocol before any run, deterministically and exactly, and outputs severity, tolerance degree, and repair. The two are complementary: TRAC's constraints can supply $\mathcal{H}$; its monitor can enforce at run time what typing certified.

	TRAC [42]	this paper
when	run time (audit, monitor, intervene)	compile time, before any run
scope	one agent's input/output trace	multiparty protocol
specification	LTL over propositions	protocol + world + goal + hazard
abstraction	language-model labelers (learned)	structured boundary (no LLM)
prediction	sampling continuations (probabilistic)	exact reachability (deterministic)
output	violation, witness, intervention	severity per branch, k^*, repair
guarantee	empirical reduction of violation rate	Theorems 7–12

Table 1 Axes on which the two approaches differ.

10 Relation to Runtime Compliance Monitoring

The closest work by goal is runtime compliance monitoring of language-model agents, of which TRAC [42] is the most developed: behavioural constraints in linear temporal logic, offline auditing of traces, online monitoring, predictive monitors that sample likely continuations, and intervening monitors that preempt a predicted violation, evaluated on text-based agent environments. We share the question and differ on every mechanism (Figure 4, Table 1).

Two differences are the deep ones. *Prediction versus projection.* TRAC must *estimate* whether a violation is coming, by sampling continuations of a black-box process. Here no estimate is needed: the guards determine the intended branch and the reachability search determines the consequence of the others, so "prediction" is exact and free of any model of the agent. The price is expressiveness: TRAC's LTL can constrain event vocabularies that never appear as protocol messages, and our hazards are predicates on a declared world. *The trusted seam.* TRAC's soundness bottoms out in the labelers that produce its propositional abstraction, and its own results show language-model temporal reasoning degrades with event distance and constraint count — which is its argument for LTL over judges. Ours bottoms out in a deterministic checker over structured payloads, with the language model confined to

authoring. The approaches compose: an LTL safety constraint is a natural source for $\mathcal{H}$, and TRAC's intervening monitor is a natural enforcer at run time of the tolerance degree typing certified statically. TRAC's finding that violations concentrate in ordering and sequencing is independent evidence that the structural layer is where the leverage is.

We note two adjacent framings. Choreographic languages with first-class agent choice mapped to games [44] own the quantitative version of "the agent may choose badly"; we are qualitative and static. Message-sequence-chart coordination for agents [45] establishes deadlock-freedom independently of agent nondeterminism; we take that independence as the starting point and ask what the nondeterminism costs.

11 What Changed, and Why

An earlier version typed *compliance* rather than *severity*: per-role modes on a trust lattice, taint propagation, staleness grades. We mechanized its metatheory and it did not survive; the audit is retained in the artifact. Its closure premise recursed at the same protocol node with a role's mode lowered and, the lattice being finite, terminated at bottom where only a quarantine rule with a *partial* residual applied: the system was vacuous, nothing containing a capability was typable (✓ `act_vacuous_with_partial_qres`, coinductively too, ✓ `act_vacuous_coinductive`). Its taint update ignored what a capability *read*, so a trusted copy laundered external data into an irreversible guard (✓ `taint_laundering_refutes_noninterference`). Its loop condition accepted cycles whose only checkpoint was a goal marker, which sanitized taint but never reset a grade (✓ `goal_only_cycle_hits_cap`). The present design has no closure premise — deviation lives in the semantics, as the budget — no taint, and no grades. We report this because the negative result produced the positive one.

12 Evaluation Plan

WIP: Measurement pending; no numbers are claimed.

(i) Tolerance degrees in the wild. Compute k^* for every corpus protocol and the public skill collection; report the distribution and, for each 0-tolerant protocol, the branch and the point-of-no-return action. Prediction: 0-tolerance is common and concentrated at irreversible tool calls. *(ii) Modularity.* Compose corpus protocols sequentially and compare wall-clock of whole-system bounded reachability against the modular derivation of Theorem 12, as k and $|\mathsf{Pred}|$ grow; the claim of §8 stands or falls here. *(iii) Predictive validity.* Run agents against the gated runtime, log in-set misselections against the guards, and check that hazard-reaching runs are exactly those exceeding k^*; a hazard within budget refutes Theorem 7's applicability to the deployment. *(iv) Repair efficacy.* Apply the repairs to 0-tolerant protocols and report the resulting degree and the cost in extra steps. Mutation testing of the classifier (seed hazards, measure detection) measures the tool rather than the theory.

13 Related Work

MPST with a fault model. Crash-stop failures [8, 9, 10], affine and exceptional sessions [11, 12], and protocol-induced recovery [13] model participants that *stop* or are cancelled; misselection is a participant that *continues, wrongly*, and none of these classify the consequence or track a goal. Asserted and refined global types [2, 6, 4, 5] supply our syntax;

monitoring [3] and blame [7] treat guard violation as an alarm rather than as an object of analysis.

Bounded faults and dead ends. Fault-tolerant planning [27, 26] bounds action failures against goal reachability; we bound selections against hazard reachability. Dead-end detection [28, 29], undoability checking [30, 31] and excuse generation [32] supply the point-of-no-return machinery; reversibility measures in reinforcement-learning safety [33] share the intuition without types.

Types equivalent to model checkers. Naik and Palsberg [21] and Kobayashi and Ong [22] set the template we instantiate; §8 says what we add. Resource-aware [23, 24] and graded [25] session types index judgments by a quantity, as we do, with the difference drawn in §6.

Agents. Runtime enforcement and provenance gating of tool calls [46, 47, 48, 49], effect-typed agent languages [50], network-enforced session types [20], and LTL trace monitoring [42, 43] act on runs or on the host language; we classify branches before any run exists (§10).

14 Limitations and Conclusion

Scope. The mechanization covers the finite fragment; recursion, and with it the interaction between loops and the budget, is future work, and a reviewer comparing scope against the mechanised subject-reduction development of [19] will be right to ask. The hazard predicate is a declared input. Three of the four repairs are argued, not mechanized. Severity is computed over the checker's widened abstraction, so Benign inherits that over-approximation while Catastrophic remains a proof. The modularity claim awaits its experiment. Everything presumes the gated architecture: an un-gated side channel falsifies the premise, not the theorem.

Conclusion. A participant that may never err may never act. The useful question is not whether it will choose wrongly — it will — but whether the protocol around it can survive the choice, and if not, what to change. Severity makes the question precise; the point of no return makes it computable with machinery the checker already has; the budget makes cascading expressible without a probability; and modular composition makes the answer a contract rather than a run. The well-typedness condition reads: *every affordable mistake is allowed, and every unaffordable one is structurally unreachable.*

References

1 K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *JACM* 63(1), 2016.

2 L. Bocchi, K. Honda, E. Tuosto, N. Yoshida. A theory of design-by-contract for distributed multiparty interactions. *CONCUR*, 2010.

3 L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, N. Yoshida. Monitoring networks through multiparty session types. *FMOODS/FORTE* 2013; *TCS* 669, 2017.

4 L. Gheri, I. Lanese, N. Sayers, E. Tuosto, N. Yoshida. Design-by-contract for flexible multiparty session protocols. *ECOOP*, 2022.

5 M. Vassor, N. Yoshida. Refinements for multiparty message-passing protocols: specification-agnostic theory and implementation. *ECOOP*, 2024.

6 F. Zhou, F. Ferreira, R. Hu, R. Neykova, N. Yoshida. Statically verified refinements for multiparty protocols. *OOPSLA*, 2020.

7 L. Jia, H. Gommerstadt, F. Pfenning. Monitors and blame assignment for higher-order session types. *POPL*, 2016.
8 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Generalised multiparty session types with crash-stop failures. *CONCUR*, 2022; *LMCS*, 2025.
9 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Designing asynchronous multiparty protocols with crash-stop failures. *ECOOP*, 2023 (Distinguished Paper).
10 K. Peters, U. Nestmann, C. Wagner. Fault-tolerant multiparty session types. *FORTE*, 2022.
11 S. Fowler, S. Lindley, J. G. Morris, S. Decova. Exceptional asynchronous session types. *POPL*, 2019.
12 N. Lagaillardie, R. Neykova, N. Yoshida. Stay safe under panic: affine Rust programming with multiparty session types. *ECOOP*, 2022.
13 R. Neykova, N. Yoshida. Let it recover: multiparty protocol-induced recovery. *CC*, 2017.
14 J. Lange, N. Yoshida. Verifying asynchronous interactions via communicating session automata. *CAV*, 2019.
15 M. R. Clarkson, F. B. Schneider. Hyperproperties. *J. Computer Security* 18(6), 2010.
16 S.-S. Jongmans, F. Ferreira. Synthetic behavioural typing: sound, regular multiparty sessions via implicit local types. *ECOOP*, 2023.
17 D. Castro-Perez, F. Ferreira, S.-S. Jongmans. *POPL*, 2026. [title to verify]
18 A synthetic reconstruction of multiparty session types. *PACMPL*, 2026. [authors to verify]
19 D. Tirore, J. Bengtson, M. Carbone. Multiparty asynchronous session types: a mechanised proof of subject reduction. *ECOOP*, 2025.
20 Larsen, Scalas, Amir, Jacobs, Wagemaker, Foster. NEST: network enforced session types. *ECOOP*, 2026.
21 M. Naik, J. Palsberg. A type system equivalent to a model checker. *ESOP* 2005; *TOPLAS* 30(5), 2008.
22 N. Kobayashi, C.-H. L. Ong. A type system equivalent to the modal mu-calculus model checking of higher-order recursion schemes. *LICS*, 2009.
23 A. Das, J. Hoffmann, F. Pfenning. Work analysis with resource-aware session types. *LICS*, 2018.
24 A. Das, D. Wang, J. Hoffmann. Probabilistic resource-aware session types. *POPL*, 2023.
25 D. Orchard, V.-B. Liepelt, H. Eades III. Quantitative program reasoning with graded modal types. *ICFP*, 2019.
26 D. Aineto, A. Gaudenzi, A. Gerevini, A. Rovetta, E. Scala, I. Serina. Action-failure resilient planning. *ECAI*, 2023.
27 C. Domshlak. Fault tolerant planning: complexity and compilation. *ICAPS*, 2013.
28 M. Steinmetz, J. Hoffmann. Towards clause-learning state space search: learning to recognize dead-ends. *AAAI*, 2016.
29 J. Hoffmann, P. Kissmann, Á. Torralba. "Distance"? Who cares? Tailoring merge-and-shrink heuristics to detect unsolvability. *ECAI*, 2014.
30 J. Daum, Á. Torralba, J. Hoffmann, P. Haslum, I. Weber. Practical undoability checking via contingent planning. *ICAPS*, 2016.
31 Non-deterministic action reversibility: complexity results. *KR*, 2025. [authors to verify]
32 M. Göbelbecker, T. Keller, P. Eyerich, M. Brenner, B. Nebel. Coming up with good excuses: what to do when no plan can be found. *ICAPS*, 2010.
33 A. M. Turner, D. Hadfield-Menell, P. Tadepalli. Conservative agency via attainable utility preservation. *AIES*, 2020.
34 B. Bittner, M. Bozzano, R. Cavada, A. Cimatti, et al. The xSAP safety analysis platform. *TACAS*, 2016.
35 L. Bocchi, J. Lange, E. Tuosto. Three algorithms and a methodology for amending contracts for choreographies. *Sci. Ann. Comp. Sci.* 22(1), 2012.
36 M. Carbone, K. Honda, N. Yoshida. Structured interactional exceptions in session types. *CONCUR*, 2008.
37 P. J. Ramadge, W. M. Wonham. Supervisory control of a class of discrete event processes. *SIAM J. Control Optim.* 25(1), 1987.

38 R. Bloem, B. Könighofer, R. Könighofer, C. Wang. Shield synthesis: runtime enforcement for reactive systems. *TACAS*, 2015.
39 M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu. Safe reinforcement learning via shielding. *AAAI*, 2018.
40 S. Basu, T. Bultan. Automated choreography repair. *FASE*, 2016.
41 I. Lanese, F. Montesi, G. Zavattaro. Amending choreographies. *WWV*, 2013.
42 P. A. Alamdari, T. Q. Klassen, S. A. McIlraith. Formal methods meet LLMs: auditing, monitoring, and intervention for compliance of advanced AI systems. *FAccT*, 2026.
43 AgentLTL: a trace-verification framework for procedural compliance in tool-using LLM agents. *arXiv:2607.02599*. [authors to verify]
44 K. Gopinathan, J. Feser, et al. Pact: a choreographic language for agentic ecosystems. *arXiv:2605.03143*, 2026. [authors to verify]
45 Provable coordination for LLM agents via message sequence charts. *arXiv:2604.17612*, 2026. [authors to verify]
46 H. Wang, C. M. Poskitt, et al. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE*, 2026.
47 E. Debenedetti et al. Defeating prompt injections by design (CaMeL). *arXiv:2503.18813*, 2025.
48 M. Costa et al. Securing AI agents with information-flow control (FIDES). *arXiv:2505.23643*, 2025.
49 T. Shi et al. Progent: securing AI agents with privilege control. *arXiv:2504.11703*, 2025.
50 ETAS: an effect-typed language for agent systems. *arXiv:2607.17780*, 2026. [authors to verify]